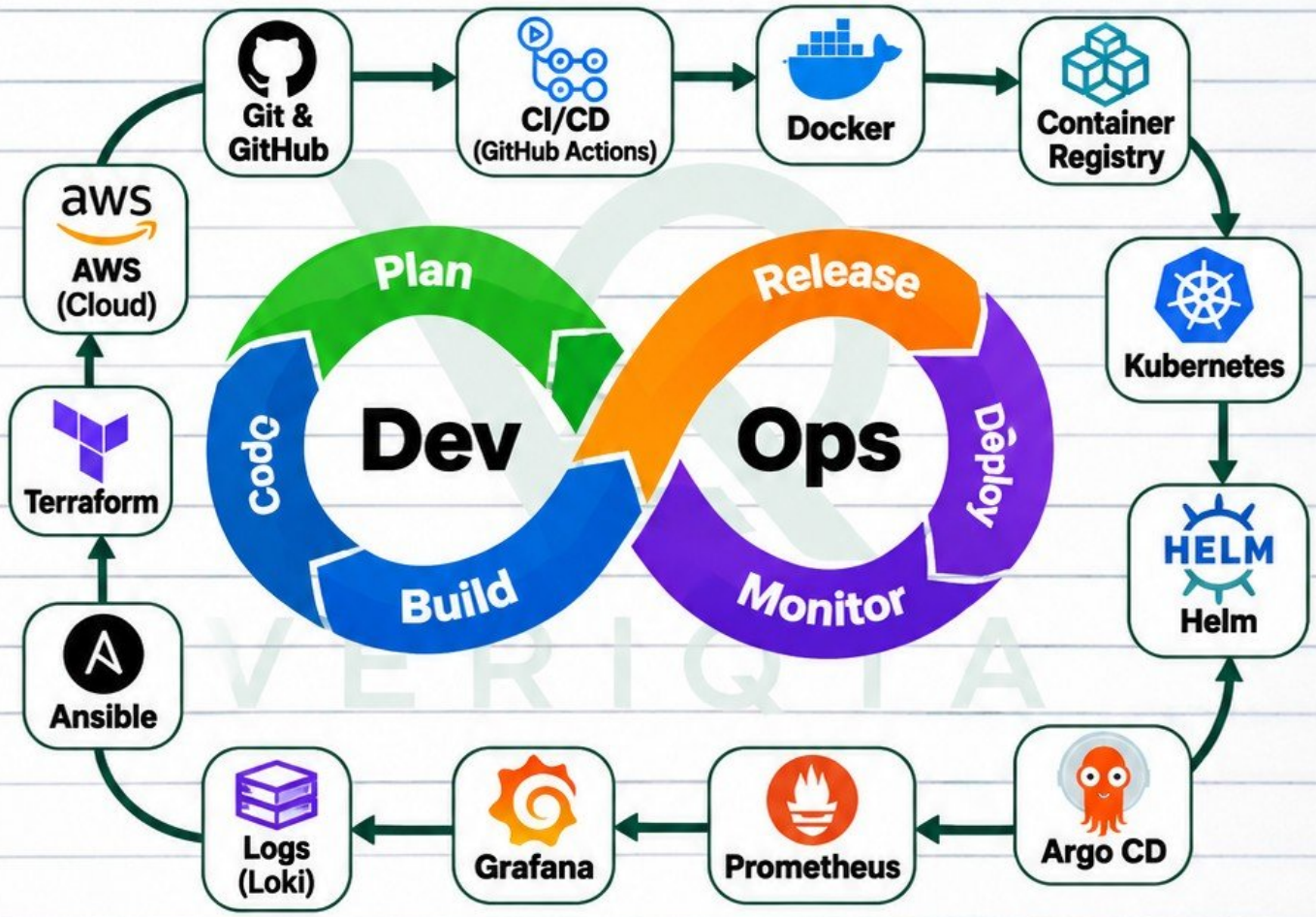
VERIQTA

# DevOps Toolchain Atlas

## A Practical Guide to the Tools, Workflow, and Real-World Integration for DevOps Engineers




- ✔ Tool Roles and Use Cases
- ✔ End-to-End DevOps Workflow
- ✔ Real Commands and Examples
- ✔ Troubleshooting and Best Practices

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

# The DevOps Toolchain: From Code to Production

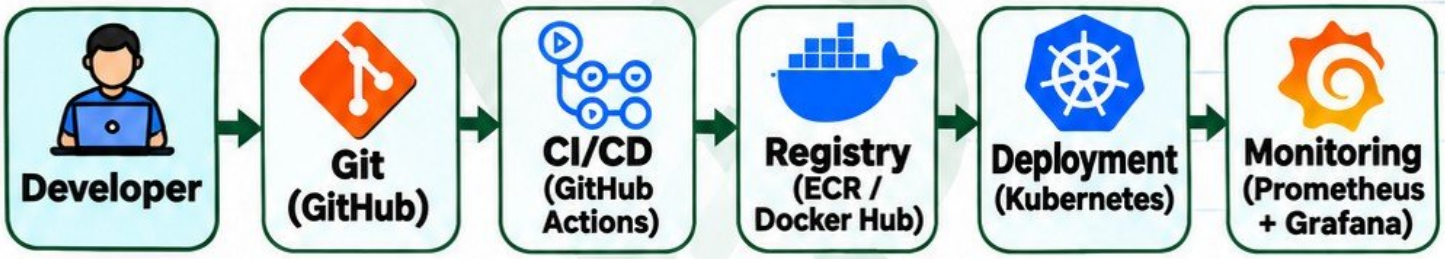
## What Is a DevOps Toolchain?

- A set of integrated tools that work together to take a code change from a developer's laptop to a running, monitored application in production.

## Why Do Teams Use Multiple Tools?

- Each tool solves a different part of the delivery process.
- Automates building, testing, packaging, deploying, and monitoring.
- Improves speed, quality, security, and reliability.

## The DevOps Toolchain Flow




## What Each Stage Produces and Consumes

| Stage | What It Produces | What the Next Stage Consumes |
|---|---|---|
| Developer | Code changes (new or updated code) | Git stores the code. |
| Git (GitHub) | Versioned source code (commit) | CI/CD pulls the code and runs checks. |
| CI/CD | Built and tested application image | Registry stores the image. |
| Registry | A versioned container image (e.g., myapp:v1.2.0) | Kubernetes pulls the image for deployment. |
| Deployment (Kubernetes) | A running application (Pods/Services) | Monitoring collects metrics, logs, and alerts. |
| Monitoring | Application health, metrics, and alerts | Engineers use it to verify, troubleshoot, and keep the app reliable. |

**Junior Engineer Insight:**
DevOps is a workflow supported by tools, not a job defined by one tool.

**Practice This:**
Pick a small app, push a change to GitHub, and run it through a CI workflow that builds a Docker image.

# The Toolchain Map: 12 Engineering Responsibilities

- The DevOps toolchain covers 12 key responsibilities from code to production.
- Each tool has a specific purpose and cannot fully replace another.
- Together, they form an end-to-end workflow that delivers, runs, and maintains reliable applications.

**1 Coding**
Write application code
VS Code
JetBrains

**2 Version Control**
Track changes and collaborate
Git
GitHub

**3 Collaboration**
Manage pull requests, reviews
GitHub
GitLab

**4 CI (Continuous Integration)**
Automatically build and test changes
GitHub Actions
Jenkins
GitLab CI

**5 Testing**
Validate code quality and functionality
pytest
JUnit
npm test

**6 Security**
Find vulnerabilities and secrets
Trivy
Semgrep
Gitleaks

**7 Packaging**
Package app and dependencies
Docker

**8 Artifact Storage**
Store versioned images and artifacts
Amazon ECR
GHCR
Docker Hub

**9 Infrastructure**
Provision cloud resources
Terraform
(Cloud IaC)

**10 Deployment**
Run applications in production
Kubernetes
(Helm/Argo CD)

**11 Configuration**
Configure servers and environments
Ansible

**12 Observability**
Monitor, collect logs and metrics
Prometheus
Grafana
Loki

## Why GitHub Actions Cannot Replace Terraform

- GitHub Actions runs CI/CD workflows.
- Terraform provisions and manages infrastructure (cloud resources).
- CI/CD cannot create VPCs, databases, or clusters.
- Teams use both: GitHub Actions for automation and Terraform for infrastructure as code.

## Why Docker Cannot Replace Kubernetes

- Docker builds and runs containers (on a single machine).
- Kubernetes runs and manages containers across multiple servers.
- Docker does not provide auto-scaling, self-healing, or load balancing.
- Teams use Docker to package applications and Kubernetes to run them in production.

## Practice: Identify the Responsible Tool

1. A build fails during the CI pipeline. Which tool is responsible?
   A. Docker  B. GitHub Actions  C. Terraform  D. Kubernetes
2. The application image is built successfully, but the deployment in the cluster fails. Which tool is most likely responsible?
   A. GitHub  B. Docker  C. Kubernetes  D. Prometheus

## Junior Engineer Insight

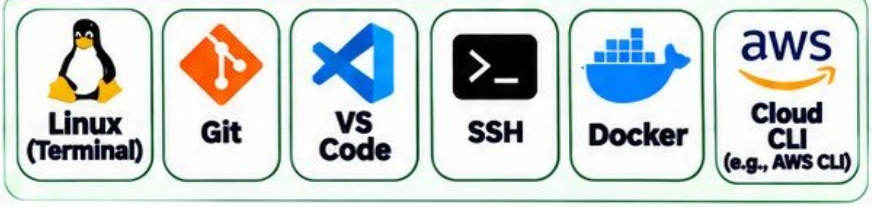
**DevOps is a workflow supported by tools,** not a job defined by **one tool.** Each tool solves a specific problem and works with others to deliver value.

# Your First DevOps Workstation

## What You'll Set Up

- A practical local environment for DevOps work, testing, and building applications before they run in CI or production.

## Key Workstation Tools




## Local vs CI vs Production Tools



**Local (Your Workstation)**

- Tools you install and use daily (Git, VS Code, Docker, CLI).
- You write code, test, build images, and try things safely before pushing to Git.



**CI / Production (Shared/Remote)**

- Tools that run in automation (e.g., GitHub Actions, Jenkins) or live environments (e.g., Kubernetes, monitoring).
- They build, test, deploy, and run your application for real users.

## Essential Commands

| Command | What It Does |
|---|---|
| git --version | Checks the installed Git version. |
| docker version | Checks Docker client and server (daemon) version. |
| ssh -V | Shows the SSH client version. |

## What a Successful Check Means

**It Proves:**

- The tool is installed and accessible on your workstation.
- The basic setup is working.

**It Does NOT Prove:**

- The tool is fully configured.
- You have correct permissions or access (e.g., to a cloud account or remote server).
- Your application will build, deploy, or run without errors.

## Troubleshooting Common Issues

**Command not found**

- The tool is not installed or not in your PATH.
- **Fix:** Install the tool and ensure it's in your PATH (e.g., sudo apt install git).

**Docker daemon unavailable**

- "Cannot connect to the Docker daemon."
- **Fix:** Start Docker (sudo systemctl start docker) and check status (sudo systemctl status docker).

**Missing permissions**

- You see "permission denied" when running a command (e.g., Docker).
- **Fix:** Use the correct user group (sudo usermod -aG docker $USER), then log out and back in.



## Security Reminder:

**Never** store cloud access keys (e.g., AWS, Azure, GCP) directly in a repository.



## Practice This:

- Run the three commands above on your Linux machine.
- If any command fails, troubleshoot and fix it before moving to the next topic.

# Git and GitHub: Where Engineering Changes Begin

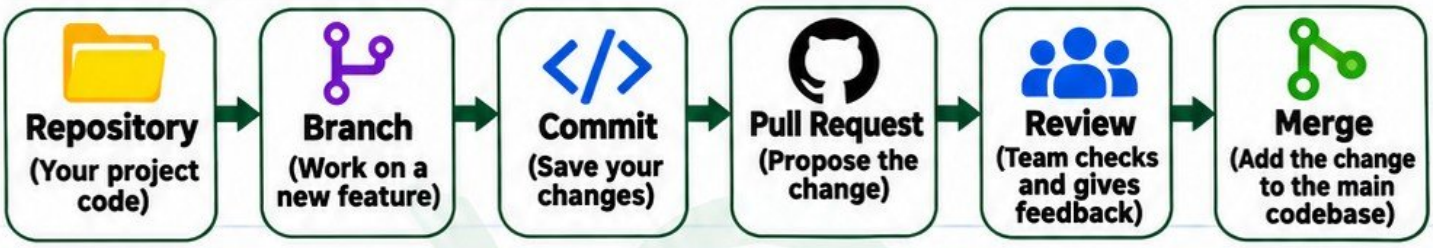
## What Are Git and GitHub?

- **Git:** A version control tool that tracks changes to your code over time.
- **GitHub:** A cloud platform that hosts Git repositories and supports team collaboration (code review, pull requests, etc.).

## Why They Matter?

- Keep a history of every change.
- Allow teams to work on features safely and in parallel.
- Enable code review before changes go live.
- Reduce the risk of breaking production.

## The Engineering Workflow




## Key Git Commands

| Command | What It Does |
|---|---|
| git status | Shows the current state of your working directory (changes, new files, etc.). |
| git switch -c feature/healthcheck | Creates and switches to a new branch named feature/healthcheck. |
| git add . | Stages your changes for a commit. |
| git commit -m "Add health check" | Saves a snapshot of your changes with a message. |

## How Code Review Protects Production



- Checks code quality.
- Catches bugs early.
- Ensures standards are followed.
- Verifies tests and security are in place.
- Prevents untested or risky changes from reaching production.

## Common Beginner Mistakes

| | |
|---|---|
| ⚠ Committing credentials: | ■ Accidentally adding secrets (e.g., API keys, cloud access keys, database passwords) to the repository. |
| ⚠ Confusing commit with push: | ■ A commit saves changes locally in your Git repository. A push uploads those changes to GitHub (the remote repository). |

## Junior Engineer Insight:

**Always use a branch, open a pull request, and get a review** before merging to the main branch.



# CI Engines: GitHub Actions, Jenkins, and GitLab CI/CD

## What Is Continuous Integration (CI)?

- An automated process that verifies code changes by building, testing, and checking for issues before deployment.
- It gives fast feedback, reduces bugs, and keeps the codebase stable.

## Popular CI Engines

**GitHub Actions**
Cloud-based CI built into GitHub. Easy to set up, YAML-driven.

**Jenkins**
Self-managed CI server. Highly flexible and extensible.

**GitLab CI/CD**
Built into GitLab. CI and CD in one platform with YAML.

## Workflow Types: Hosted vs Self-Managed

| Type | What It Means | Examples |
|---|---|---|
| Hosted (Managed) | Runs in the provider's infrastructure. No server setup or maintenance. | GitHub Actions<br>GitLab CI (SaaS) |
| Self-Managed | Runs on your own servers or infrastructure. More control and customization. | Jenkins<br>GitLab (Self-Hosted) |

## The CI Pipeline Flow

Pull Request (Code Change) → Build (Compile and package) → Test (Run automated tests) → Scan (Security and quality checks) → Result (Pass ✓ or Fail ✗)

## Pipeline Basics: YAML, Jobs, Steps, Runners, Artifacts

| | |
|---|---|
| Workflow YAML | A configuration file (e.g., .github/workflows/ci.yml) that defines the pipeline. |
| Job | A group of related tasks (e.g., build, test, scan) that run in a runner. |
| Step | An individual action inside a job (e.g., checkout code, install dependencies, run tests). |
| Runner | The machine (cloud or self-hosted) that executes your jobs and steps. |
| Build Artifact | A file produced by the pipeline, such as a compiled app, Docker image, or test report. |

## Example: GitHub Actions (YAML)

```yaml
name: ci
on: [pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
```

## Troubleshooting

**Problem:** A test passes locally but fails in CI.

### Common Causes and Fixes

| Possible Cause | How to Fix |
|---|---|
| Missing environment variable | Add it in CI secrets or pipeline config (e.g., env: or GitHub Secrets). |
| Different tool versions | Use a specific version in the pipeline (e.g., Node, Python, Java). |
| Missing dependencies | Ensure installation in the pipeline (e.g., npm install or pip install -r requirements.txt). |

## Practice: Identify the Failing Stage

A CI pipeline shows:

Build ✓ → Test ✗ → Scan → Deploy

Which stage failed?

A. Build
B. Test
C. Scan
D. Deploy

**Answer: B – Test**

# Build and Test Tools: Proving the Application Works

## Why Testing in CI Matters

- Passing CI is more than just compiling the code.
- It checks that the application builds, works, and meets expected behavior.
- It catches defects early, reduces production risk, and keeps releases reliable.

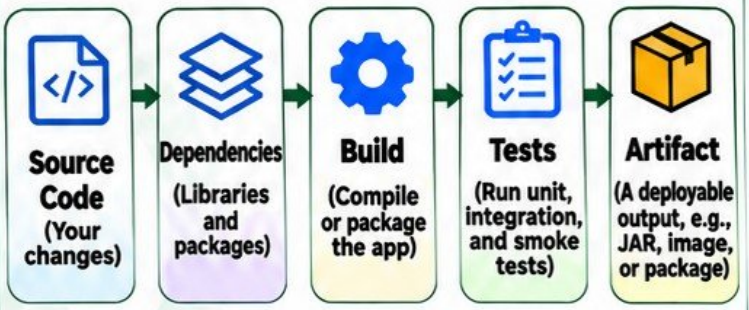
## Common Build & Test Tools

| Language | Tool | Command | What It Does |
|---|---|---|---|
| Python | pytest | pytest | Runs automated tests for Python apps (unit, integration, etc.). |
| Java | Maven | mvn test | Builds the project, resolves dependencies, and runs tests. |
| JavaScript | npm | npm test | Runs the project's automated tests (e.g., Jest, Mocha). |

## Types of Tests

| Test Type | What It Tests | Example |
|---|---|---|
| Unit Test | Tests a small piece of code (e.g., a single function or class). | Test a user registration function. |
| Integration Test | Tests how multiple components work together (e.g., API + DB). | Create a user and verify it is saved in the database. |
| Smoke Test | A quick end-to-end check that the main features work after a build or deployment. | Start the app and verify the health endpoint returns 200. |

## The Build and Test Flow




## Stop the Release on Test Failure




- If tests fail, the pipeline must stop.
- This prevents broken, unstable, or unverified code from reaching production.
- Teams fix the issue, commit a new change, and run the pipeline again.

## Example CI Test Output

```
$ pytest
============================
collected 5 items
test_user.py ....          [ 80%]
test_payment.py F          [100%]
============================
FAILED: 1 test failed
Pipeline: Failed (Deployment blocked)
```

## Production Connection

- Tests reduce deployment risk by catching issues early.
- They improve confidence in a release but do NOT guarantee a defect-free application.

## Key Takeaways

- Use the right test typs for the right purpose.
- Automate tests in your CI pipeline.
- A failing test should block deployment.
- Testing lowers risk, but real user monitoring in production still matters.

# Security Tools: Finding Problems Before Deployment

## Why Security Scanning Matters

- Finds security issues early in the development cycle.
- Reduces the risk of deploying vulnerable code, dependencies, or container images.
- Catches common problems such as insecure code, known CVEs, and exposed secrets.

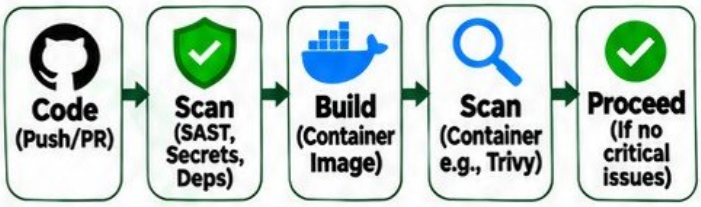
## Key Security Scanning Types

| | |
|---|---|
| **SAST (Static Analysis)** | Scans your source code for security weaknesses (e.g., Semgrep). |
| **Dependency Scanning** | Checks third-party libraries for known vulnerabilities (e.g., Dependabot). |
| **Secret Scanning** | Finds exposed credentials (e.g., API keys, tokens) in code (e.g., Gitleaks). |
| **Container Scanning** | Scans container images for vulnerabilities (e.g., Trivy). |

## Common Security Tools and What They Do

| Tool | Focus Area | What It Inspects |
|---|---|---|
| Semgrep | SAST (Code Scan) | Your source code for security issues (e.g., unsafe input, hardcoded secrets). |
| Trivy | Container Scanning | Container images and OS packages for known CVEs (vulnerabilities). |
| Gitleaks | Secret Scanning | Source code and files for exposed secrets (e.g., API keys, database passwords). |
| Dependabot | Dependency Scanning | Your project dependencies (e.g., npm, Maven, pip) for known security issues. |

## Where Each Tool Fits in the Pipeline




## Example: Scan a Container Image

```
$ trivy image myapp:1.0
myapp:1.0 (linux/amd64)
 CRITICAL    HIGH    MEDIUM    LOW
    1          3        7       12
```

## Troubleshooting: A Critical Vulnerability Blocks a Release

| Step | Action |
|---|---|
| 1 Identify the issue | Check the scan report for the CVE, severity, and affected package. |
| 2 Fix or mitigate | Update the vulnerable dependency or use a patched base image. |
| 3 Rebuild and rescan | Rebuild the image and run the scan again (e.g., trivy image myapp:1.0). |
| 4 Proceed or block | Deploy only if critical/high issues are resolved or accept a documented risk with mitigation. |

## Security Reminder:

- Scanning is one layer of protection, not proof that an application is fully secure.

## Key Takeaways

- Use the right tool for the right scope: Semgrep (code), Gitleaks (secrets), Dependabot (dependencies), Trivy (containers).
- Fix critical issues before deployment.
- Combine scanning with secure coding, access control, and monitoring in production.

# Docker: Turning an Application into a Deployable Image

## What Is Docker?

- A platform that packages your application and all its runtime dependencies into a single, portable image.
- The image can run anywhere (e.g., your laptop, a CI pipeline, or a production cluster) as a container.

## Why Package the Application?

- ✓ Ensures the app runs the same in development, testing, and production.
- ✓ Includes everything the app needs (e.g., runtime, libraries, config).
- ✓ Reduces "it works on my machine" issues.
- ✓ Makes deployments faster, repeatable, and more reliable.

## The Docker Workflow

**Source Code** (Your application files) → **Dockerfile** (Instructions to build the image) → **Image** (A packaged version of the app, e.g., myapp:1.0) → **Container** (A running instance of the image)

## Key Docker Commands

| Command | What It Does |
|---|---|
| `docker build -t myapp:1.0 .` | Builds a Docker image from the Dockerfile in the current directory and tags it as myapp:1.0. |
| `docker run --rm -p 8080:8080 myapp:1.0` | Runs a container from the image, maps port 8080 on your machine to port 8080 in the container, and removes it after it stops. |

## A Simple Dockerfile Example

```
# Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python", "app.py"]
```

## Troubleshooting Common Issues

| Issue | Possible Cause and Fix |
|---|---|
| ⚠ Container exits immediately | ■ The main process ends (e.g., wrong CMD or the app crashes).<br>■ **Fix:** Check the logs: docker logs <id> and verify the start command. |
| ⚠ Application cannot be reached | ■ Wrong port mapping, app not listening, or firewall rule.<br>■ **Fix:** Confirm the app listens on 0.0.0.0:8080 and the port is correctly mapped with -p 8080:8080. |

## ✗ Beginner Mistake:

Assuming a successful image build means the application will run correctly. A build only creates the image; you must run and test the container.

## ✓ Production Connection

- Use Docker to build consistent, tested releases in CI/CD.
- Push the image to a registry (e.g., ECR, Docker Hub), then deploy to Kubernetes.

## Key Takeaway

- Docker turns your application into a portable, repeatable unit.
- Build → Test → Run → Push → Deploy.

# Container Registries: Where Deployment Images Live

## What Is a Container Registry?

- A storage service for container images (e.g., your application images).
- Keeps versioned images in repositories for use in CI/CD and deployment.
- Provides secure access, version control, and distribution.

## Popular Container Registries

| Tool | Type | Best For |
|---|---|---|
| Amazon ECR | Cloud (AWS) | AWS workloads, IAM integration, high availability. |
| GitHub Container Registry | Cloud (GitHub) | Tightly integrated with GitHub and GitHub Actions. |
| Docker Hub | Cloud (Public/Private) | General use, large community, public and private images. |
| Harbor | Self-Managed (On-Prem/Cloud) | Enterprise use with access control, vulnerability scanning, and image policies. |

## Key Concepts

| | |
|---|---|
| **Repository** | A collection of related container images (e.g., veriqta/api). |
| **Tag** | A human-readable label for an image version (e.g., myapp:1.0, latest). |
| **Digest (Immutable)** | A unique, content-based ID (e.g., sha256:ab23...) that does not change. |
| **Authentication** | Verifies your identity (e.g., AWS IAM, GitHub token, Docker Hub login). |
| **Image Pull** | Downloading an image from a registry to run it (e.g., in Kubernetes). |

## The Deployment Flow

**CI Builds Image** (Builds and tags your container image) → **Registry Stores Image** (Stores the image in a repository with a tag and digest) → **Kubernetes Pulls Image** (Downloads the image from the registry and runs it as a container)

## Why Digests Are Stronger Than Tags

| Aspect | Tag (e.g., v1.0) | Digest (e.g., sha256:ab23...) |
|---|---|---|
| Mutability | Can change (overwritten). | Immutable (never changes). |
| Identity | Not guaranteed to be the same image. | Uniquely identifies one exact image. |
| Release Safety | Risk of using a different image. | Ensures the exact, tested image is deployed. |

## Troubleshooting Common Issues

| Issue | Likely Cause | Fix |
|---|---|---|
| Unauthorized image push | Missing or incorrect authentication/permissions (e.g., IAM or registry token). | Log in and ensure you have push (write) permission to the repository. |
| Kubernetes ImagePullBackOff | Kubernetes cannot pull the image (e.g., wrong image name/tag, no permission, or image does not exist). | Verify the image name, tag/digest, and registry credentials (e.g., imagePullSecret). |

## Practice: Registry Permission

A Kubernetes deployment needs to download an image from a private ECR repository. Which registry permission is required?

A Read (Pull) B Write (Push) C Delete D Admin

**Answer: A – Read (Pull)**

## Key Takeaway

- Registries store and version your container images securely.
- Use image digests for reliable, repeatable, and auditable releases.

# Terraform: Building Infrastructure with Code

## What Is Terraform?

- A popular Infrastructure as Code (IaC) tool that lets you define, create, and manage infrastructure using code.
- Uses a declarative approach: you describe the desired infrastructure, and Terraform makes it match that state.

## Key Terraform Concepts

| Concept | What It Means |
|---|---|
| Provider | A plugin that connects Terraform to a platform (e.g., AWS, Azure, GCP). |
| Resource | An infrastructure object you want to create (e.g., VPC, EC2, S3, EKS). |
| State | A record of what Terraform has created and is currently managing. |
| Plan | Shows the changes Terraform will make (add, change, or destroy) before applying. |
| Apply | Executes the approved changes to create or update the infrastructure. |

## Common Terraform Commands

| Command | Purpose |
|---|---|
| terraform init | Downloads provider plugins and sets up the working directory. |
| terraform validate | Checks the configuration for syntax and basic errors. |
| terraform plan | Previews the changes Terraform will make to your infrastructure. |

## The Terraform Workflow

Configuration (.tf files) (Describe the desired infrastructure) → Plan (Preview the changes Terraform will make) → Approval (Review the plan and confirm it is safe) → Infrastructure (Created or updated in the cloud e.g., AWS)

## Example: Terraform Configuration (AWS)

```
provider "aws" {
  region = "eu-west-1"
}
resource "aws_instance" "web" {
  ami           = "ami-0123456789abcdef0"
  instance_type = "t3.micro"
  tags = {
    Name = "veriqta-web"
  }
}
```

## Why Remote State, Locking, and Protected Credentials Matter

| | |
|---|---|
| Remote State | Stores the Terraform state in a shared and secure location (e.g., AWS S3) so the team uses the same source of truth. |
| Locking | Prevents multiple people from running changes at the same time, which can cause conflicts and inconsistent infrastructure. |
| Protected Credentials | Keep cloud access keys secure using environment variables, IAM roles, or a secrets manager (e.g., AWS Secrets Manager), not in code or Git. |

## Production Awareness

- Never apply an unreviewed infrastructure change.
- Always review the plan, understand the impact, and get the necessary approval.
- Treat infrastructure changes like code changes in a pull request.

## Key Takeaways

- Terraform lets you define and manage infrastructure with code (IaC).
- Use init → validate → plan → apply in a controlled flow.
- Use remote state, locking, and protected credentials for safe, team-based operations.
- Infrastructure changes should be reviewed, approved, and auditable.

# Cloud Platforms: AWS, Azure, and Google Cloud

## What Cloud Providers Supply

- On-demand, scalable infrastructure and managed services.
- Global regions, high availability, security, and reliability.
- Core services for compute, storage, networking, databases, and monitoring.

## What Engineers Still Manage

- Application design, configuration, and deployments.
- Access control, security policies, and network rules.
- Cost, performance, monitoring, and operational reliability.
- Data, backups, and business continuity.

## Major Cloud Providers

**AWS**
(Broadest services, strong ecosystem)

**Azure**
(Enterprise integration, hybrid, and Microsoft ecosystem)

**Google Cloud**
(Modern, data, AI/ML, and analytics strengths)

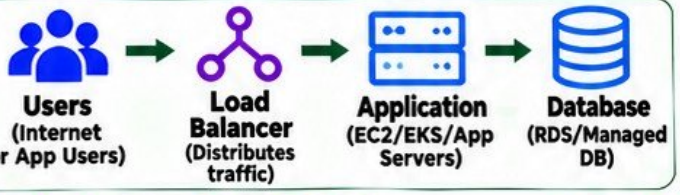
## Example AWS Services (Common Use)

| EC2 | VPC | S3 | IAM | RDS | EKS |
|---|---|---|---|---|---|
| Run virtual servers (VMs). | Isolated network for your resources. | Scalable object storage for files. | Manage users, roles, and access. | Managed relational database (e.g., MySQL, PostgreSQL). | Managed Kubernetes for running containers. |

## Key Cloud Service Map

| Area | What It Provides | AWS Examples |
|---|---|---|
| Compute | Virtual servers to run apps. | EC2 (VMs), EKS (Kubernetes) |
| Networking | Secure, isolated connectivity. | VPC, Subnets, Route Tables, SG |
| Storage | Durable, scalable data storage. | S3 (Object), EBS (Block), EFS (File) |
| Identity | Access and permissions control. | IAM (Users, Roles, Policies) |
| Databases | Managed database services. | RDS (Relational), Aurora |
| Monitoring | Metrics, logs, and alerts for operations. | CloudWatch (Metrics, Logs, Alarms) |

## A Simple Cloud Architecture




## Public vs Private Networking

| | |
|---|---|
| Public Network | ■ Accessible from the internet.<br>■ Used for entry points (e.g., load balancers, web servers). |
| Private Network | ■ Not directly accessible from the internet.<br>■ Used for internal resources (e.g., databases, internal services). |

## Basic Access Controls

- **IAM:** Control who can access what (least privilege).
- **Security Groups:** Allow or block traffic (e.g., allow port 80/443).
- **Network ACLs:** Extra network filtering at the subnet level.
- **VPC Design:** Separate public and private subnets for security.

## Troubleshooting: App Is Healthy but Unreachable

| Likely Cause | What to Check and Fix |
|---|---|
| Security group rule | ■ Ensure the load balancer or instance allows traffic on the correct port (e.g., 80/443). |
| Network ACL (NACL) | ■ Check if the subnet ACL is blocking inbound or outbound traffic. |
| Wrong subnet/routing | ■ Confirm the instance is in the correct subnet and has a route to the internet (or load balancer). |
| Load balancer target | ■ Verify the target is registered and marked Healthy in the load balancer. |

## Key Takeaways

- Cloud providers give you powerful building blocks, not a fully managed application.
- Design networks, access, and monitoring correctly from the start.
- If an app is healthy but unreachable, check security groups, NACLs, routes, and load balancer targets.

# Ansible: Configuring Servers Consistently

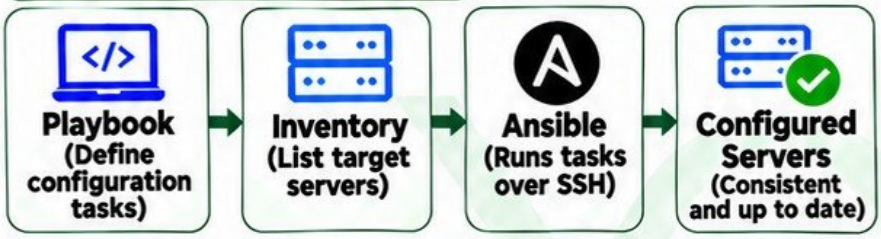
## Provisioning vs Configuration

- **Provisioning:** Creates the infrastructure (e.g., EC2, VMs, networks, storage) — usually with tools like Terraform or cloud consoles.
- **Configuration:** Sets up and manages the operating system and software (e.g., install Nginx, create users, set configs) — using tools like Ansible.

## Key Ansible Concepts and Tools

| Tool / Term | What It Does |
|---|---|
| Ansible | Automation tool for configuring and managing servers over SSH. |
| SSH | Secure remote access used by Ansible to connect to managed servers. |
| Inventory | A file that lists the target servers (e.g., web, app, db). |
| Playbook | A YAML file that defines the automation tasks to run. |
| Module | Reusable Ansible functions (e.g., package, service, copy, template). |

## The Ansible Workflow




## What Is Idempotence?

- Ansible runs can be repeated safely without causing unwanted changes.
- It only makes changes if the target is not already in the desired state.
- This keeps servers consistent and reduces configuration drift (differences between servers).

## Example: Install and Configure Nginx

site.yml (Playbook)

```
---
- name: Install and configure Nginx
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      ansible.builtin.package:
        name: nginx
        state: present
    - name: Ensure Nginx is running and enabled
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: yes
```

## Run the Playbook

```
$ ansible-playbook -i inventory.ini site.yml
```

## Example Inventory (inventory.ini)

```
[web]
web01 ansible_host=10.0.1.10
web02 ansible_host=10.0.1.11
```

## What the Playbook Does

- Installs Nginx on the servers in the [web] group.
- Starts the Nginx service.
- Enables it to start automatically at boot.

## Troubleshooting Common Issues

| Issue | Likely Cause | Fix |
|---|---|---|
| SSH authentication failure | Wrong key, user, or host not reachable. | Verify SSH access: ssh user@host. Ensure the correct SSH key is used in the inventory. |
| Missing privilege escalation | The playbook needs admin (sudo) rights but it's not allowed. | Add become: yes in the playbook and ensure the user has sudo permission. |

## Production Awareness

- Use version-controlled playbooks (e.g., in Git).
- Test in a staging environment before running in production.
- Keep inventory and credentials secure (e.g., Ansible Vault or a secret manager).

## Key Takeaway

- Ansible gives you repeatable, consistent, and scalable configuration.
- Use idempotent playbooks to reduce drift and keep servers reliable.

# Kubernetes: Running Containers Across Servers

## What Is Kubernetes?

- An open-source platform that runs and manages containerized applications across multiple servers (nodes).
- It automates deployment, scaling, health checks, and recovery to keep your applications available and reliable.

## Key Kubernetes Components

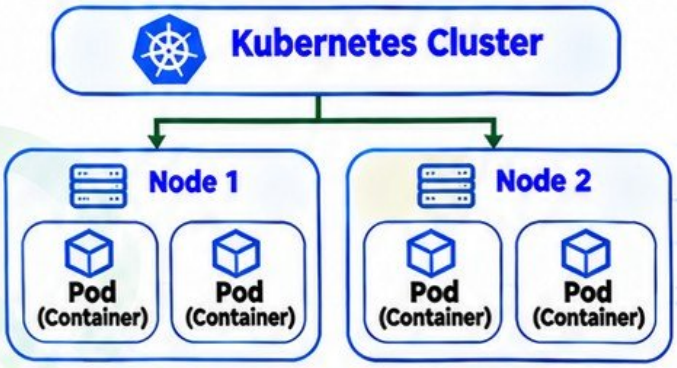
| Component | What It Is / Does |
|---|---|
| Cluster | The whole Kubernetes environment (control plane + worker nodes). |
| Node | A server (VM or instance) that runs Pods. |
| Pod | The smallest deployable unit that runs one or more containers. |
| Deployment | Manages Pods and keeps the desired number of replicas running. |
| Service | Provides a stable network endpoint to access Pods (e.g., inside or outside the cluster). |

## How Kubernetes Keeps Applications Running




- ✓ You define a desired number of replicas (e.g., 3).
- ✓ Kubernetes ensures that number is always running.
- ✓ If a Pod fails, it automatically creates a new one to maintain the desired count.

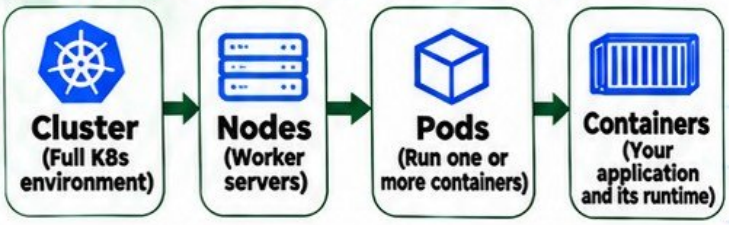
## Kubernetes Architecture (Overview)




## Common kubectl Commands

| Command | What It Does |
|---|---|
| kubectl get pods | Lists all Pods and their status (Running, Pending, CrashLoopBackOff, etc.). |
| kubectl get deployments | Shows Deployments and the number of desired and available replicas. |
| kubectl get services | Lists Services and their cluster IP, port, and type (ClusterIP, NodePort, LoadBalancer). |

## Cluster → Nodes → Pods → Containers




## Troubleshooting Common Issues

| Issue | Likely Cause | What to Check and Fix |
|---|---|---|
| ⚠ Pod is Pending | ■ Not enough resources (CPU/Memory).<br>■ Scheduling constraints (e.g., node selector, taints). | ■ Run kubectl describe pod <pod-name> to see the reason.<br>■ Increase resources or adjust the schedule rules. |
| ⚠ Pod is CrashLoop BackOff | ■ The container starts but keeps crashing (e.g., bad config, missing dependencies, app error). | ■ Check logs: kubectl logs <pod-name><br>■ Fix the application, config, or environment issue and redeploy. |

## ✕ Beginner Mistake

**Confusing a Pod with a Deployment.**

- A Pod is a running instance of your container(s).
- A Deployment manages one or more Pods and keeps the desired number of replicas running.

## Key Takeaway

- Kubernetes runs, scales, and heals your containerized applications.
- Use kubectl to check the health of Pods, Deployments, and Services.
- Deployments keep the right number of Pods running, even when failures happen.

# Helm: Packaging Kubernetes Applications

## Why Use Helm Charts?

- Packages and manages Kubernetes resources (Deployments, Services, Ingress, etc.) as a single, reusable unit.
- Reduces complexity compared to maintaining many disconnected YAML files.
- Makes applications easier to configure, version, and deploy across environments (dev, staging, production).

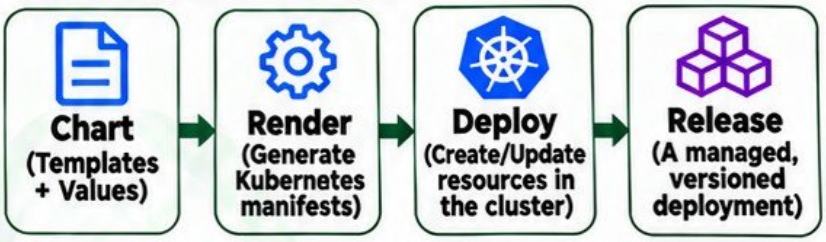
## Key Helm Terms

| Term | What It Means |
|---|---|
| Chart | A package that contains all the files needed to deploy an application. |
| Templates | Kubernetes YAML files with variables (e.g., {{ .Values.image.tag }}). |
| Values | A configuration file (values.yaml) that provides environment-specific settings. |
| Release | A deployed instance of a chart in a Kubernetes cluster. |
| Revision | A version of a release (each upgrade creates a new revision). |

## Helm Chart Structure (Example)

```
mychart/
  Chart.yaml      # Chart metadata
  values.yaml     # Default values
  templates/
    deployment.yaml
    service.yaml
    ingress.yaml
  README.md       # How to use the chart
```

## How Helm Works




## Common Helm Commands

| Command | Purpose |
|---|---|
| helm lint ./mychart | Checks the chart for syntax and best practice issues. |
| helm template myapp ./mychart | Renders the Kubernetes manifests without deploying (helps you review the output). |

## Different Values for Different Environments

values-dev.yaml

```
replicaCount: 1
image:
  tag: "dev"
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
```

values-prod.yaml

```
replicaCount: 3
image:
  tag: "1.0.0"
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

## Troubleshooting Common Issues

| Issue | Likely Cause | Fix |
|---|---|---|
| Invalid manifests | Incorrect values (e.g., wrong type, missing required field). | Run helm lint and helm template to identify the error. |
| Misconfigured workload | Wrong image tag, replica count, or resource settings. | Validate your values.yaml and confirm the generated manifests are correct. |

## Production Awareness

- Always render and review the manifests before deployment (helm template).
- Use versioned chart releases (e.g., 1.0.0, 1.0.1) for traceability.
- Test the chart in a non-production environment first.

## Key Takeaway

- Helm makes Kubernetes deployments simpler, reusable, and consistent.
- Use different values for each environment and always review the rendered manifests before deploying.

# Argo CD: Deploying Through GitOps

## What Is GitOps?

- A modern operational approach where Git is the single source of truth for your deployment configuration.
- You declare the desired state (e.g., Kubernetes manifests or Helm charts) in Git, review changes (e.g., via pull requests), and let a tool like Argo CD apply and keep the environment in sync.

## Key Tools and Roles

| Tool | Role in GitOps |
|---|---|
| Argo CD | Watches Git, compares the desired state with the cluster, and deploys/keeps them in sync. |
| Git Repository | Stores the application manifests, Helm charts, and configuration (declarative desired state). |
| Kubernetes | Runs the application (the target environment). |
| Helm | Packages and templates the Kubernetes configuration for easier management. |

## How GitOps with Argo CD Works

**Git Change**
(Update manifests/ Helm chart in Git and merge)

→

**Argo CD Reconciliation**
(Detects the change, compares, and syncs)

→

**Kubernetes**
(Applies the change and runs the updated application)

## Key GitOps Concepts

| Concept | Meaning |
|---|---|
| Desired State | The configuration in Git (what the application should look like). |
| Live State | The actual state currently running in Kubernetes. |
| Synchronization | The process of making the live state match the desired state. |
| Drift | A difference between the desired state (Git) and the live state (cluster). |

## Example: Argo CD Application (Helm)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: myapp
spec:
  source:
    repoURL: https://github.com/veriqta/platform-app.git
    targetRevision: main
    path: charts/myapp
  destination:
    server: https://kubernetes.default.svc
    namespace: myapp
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

## Common Argo CD Status and Meaning

| Status | What It Means |
|---|---|
| Synced | The live state matches the desired state in Git. |
| OutOfSync | The live state is different from the Git configuration (drift exists). |
| Degraded | The application is deployed, but one or more resources are unhealthy (e.g., Pods crashing, failed rollout). |
| Progressing | A deployment or sync is still in progress (e.g., new Pods are being creater). |

## Troubleshooting Common Issues

| Issue | Likely Cause | Fix |
|---|---|---|
| OutOfSync | Manual changes in the cluster or a different configuration. | Review the diff in Argo CD and sync to the Git state (or revert the manual change). |
| Degraded | Application is not healthy (e.g., Pod crash, bad image, or misconfiguration). | Check the application events and Pod logs, fix the issue (e.g., image, config, probes), then sync again. |

## Security Reminder

- Do NOT commit plaintext secrets (e.g., passwords, API keys) to Git.
- Use a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault), Kubernetes Secrets, or a secrets plugin (e.g., External Secrets Operator).
- Apply least privilege access for Git, Argo CD, and the Kubernetes cluster.

## Key Takeaway

- GitOps with Argo CD keeps deployments consistent, reviewable, and auditable.
- Use Git as the source of truth, keep environments in sync, and protect your secrets.

# Networking Tools: Following an Application Request

## Key Networking Concepts

| | |
|---|---|
| **DNS** | Translates a domain name (e.g., example.com) to an IP address. |
| **TCP Ports** | Network entry points for services (e.g., 80 for HTTP, 443 for HTTPS). |
| **HTTP** | The protocol used by browsers and apps to request and receive data (e.g., GET, POST). |
| **TLS** | Encrypts the connection (HTTPS) and verifies the server with a certificate. |
| **Load Balancer** | Distributes incoming traffic across multiple application instances. |
| **Ingress** | A Kubernetes resource that exposes services to external traffic (routing, TLS, host rules). |

## Networking Tools and Uses

| Tool | What It Does |
|---|---|
| **dig** | Queries DNS to resolve domain names (e.g., dig example.com). |
| **curl** | Sends HTTP/HTTPS requests and shows the response (e.g., headers, status code). |
| **ss** | Shows active network connections, listening ports, and process details (e.g., ss -tulpn). |
| **openssl** | Checks TLS/SSL details, such as the certificate and handshake (e.g., openssl s_client -connect example.com:443). |

## Common curl Command

```
$ curl -I https://example.com
```

- Sends a HEAD request (headers only) to the URL.
- Helps verify that the site is reachable, the TLS is working, and the response status code (e.g., 200, 301, 503).

## End-to-End Request Flow

**Browser** (User request (e.g., https://app.com)) → **DNS** (Resolves domain to an IP) → **Load Balancer** (Receives traffic and distributes it) → **Ingress** (Routes traffic inside the cluster (e.g., by host/path) → **Service** (Stable access to the application (Pod group)) → **Pod** (Runs the application container)

## Troubleshooting: Common Request Failures

| Symptom / Error | Likely Cause | What to Check / Fix |
|---|---|---|
| **DNS failure** (e.g., "could not resolve host") | Domain not found or wrong DNS record. | Check DNS with dig (e.g., dig example.com). |
| **Connection refused** (e.g., "Connection refused") | Service not listening, wrong port, or firewall blocking access. | Verify the service is running (ss -tulpn), check the port, and security group/firewall rules. |
| **TLS error** (e.g., "certificate verify failed") | Invalid, expired, or misconfigured TLS certificate. | Check the certificate with openssl s_client -connect example.com:443 and fix/renew it. |
| **HTTP 503** (Service Unavailable) | Application not ready, no healthy Pods, or backend failure. | Check Pod health (kubectl get pods), service endpoints, and application logs. |

## Junior Engineer Insight

- A failed request does NOT automatically mean the application code is broken.
- The issue can be DNS, network connectivity, TLS, load balancer, ingress, service, or the application itself.
- Follow the request path step by step (from DNS to Pod) to find the real root cause.

## Key Takeaway

- Use the right networking tools to trace a request from the browser to the Pod.
- Understand DNS, ports, TLS, and load balancing to troubleshoot faster and correctly.

# Prometheus and Grafana: Understanding System Health

## What Are Prometheus and Grafana?

**Prometheus**
- A time-series monitoring tool that collects (scrapes) metrics from your applications, servers, and infrastructure.

**Grafana**
- A visualization tool that reads data from Prometheus and turns it into dashboards, charts, and alerts.

## Key Concepts

| Term | Meaning |
|---|---|
| Metrics | Numerical data about your system (e.g., requests, errors, CPU, memory). |
| Scraping | The process of Prometheus periodically collecting metrics from targets (endpoints). |
| Time Series | Metrics stored with a timestamp (e.g., CPU usage over time). |
| Dashboards | Visual views of key metrics in Grafana (e.g., traffic, errors, latency). |
| Alerting | Rules that watch metrics and notify you (e.g., high error rate or high CPU). |

## Key Application Metrics to Watch

| Metric | What It Tells You |
|---|---|
| Request Rate | How many requests your application is handling (e.g., requests per second). |
| Error Rate | The percentage of failed requests (e.g., HTTP 5xx or 4xx). |
| Latency | How long requests take to complete (e.g., p50, p95, p99). |
| Resource Saturation | How busy your system resources are (CPU, memory, disk, network). High and sustained usage can lead to performance issues. |

## How It Works (End-to-End)

**Application** (Exposes metrics endpoint, e.g., /metrics) → **Prometheus** (Scrapes and stores metrics as time series) → **Grafana** (Visualizes metrics with dashboards and alerts)

## Why a Running Process Can Still Fail

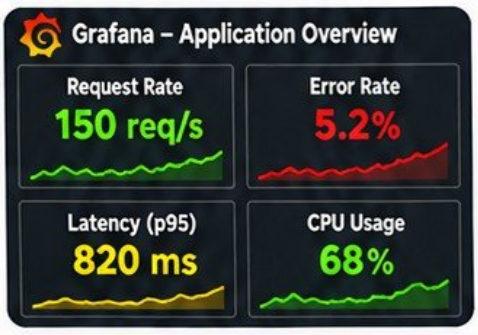
- A process can be running (e.g., the web server is alive) but still serve a failing application due to:
  - ✅ Application errors (e.g., database connection failures).
  - ✅ High error rate (e.g., HTTP 5xx responses).
  - ✅ Performance issues (e.g., high latency or resource exhaustion).
  - ✅ Misconfiguration (e.g., wrong environment or dependency).

## Example: Useful Prometheus Queries

```
# Request rate (requests per second)
  rate(http_requests_total[5m])
# Error rate (HTTP 5xx percentage)
  rate(http_requests_total{status=~"5.."}[5m]) /
  rate(http_requests_total[5m]) * 100
# Latency (95th percentile)
  histogram_quantile(0.95, rate(http_request_duration_
  seconds_bucket[5m]))
```

## Practice: Investigate Rising HTTP 5xx Errors

1. Check Grafana: Is the 5xx error rate increasing after the latest deployment?
2. Drill down: Which service or endpoint is returning the errors?
3. Correlate: Check related metrics (latency, CPU, memory, database errors).
4. Validate: Review recent changes (e.g., new release, config, or dependency).
5. Fix and verify: Roll back or patch, then confirm error rate returns to normal.




## Production Tips

- Define clear alert thresholds (e.g., error rate > 2%, latency > 1s).
- Monitor key metrics together (rate, errors, latency, and resources).
- Review dashboards after every major deployment.

## Key Takeaway

- Prometheus collects metrics, and Grafana turns them into insights and alerts.
- Use metrics to detect issues early, understand impact, and keep your services healthy.

# Logs and Traces: Investigating Production Failures

## What Are Logs and Traces?

- **Logs:** Time-stamped event records that show what happened in your system (e.g., requests, errors, warnings).
- **Traces:** The end-to-end path of a single request as it moves through multiple services (e.g., API → backend → database), including timing at each step.

## Key Tools and Their Use

| Tool | What It Does |
|---|---|
| Loki | Log aggregation for container and Kubernetes workloads (lightweight). |
| Elasticsearch | Searches, stores, and analyzes large volumes of logs. |
| OpenTelemetry | A standard for collecting, processing, and exporting traces, metrics, and logs. |
| Jaeger | Distributed tracing to visualize a request across multiple services, with timing and dependencies. |

## The Role of a Request ID

- ✅ A request ID is a unique identifier (e.g., `req-7f3a2e`) added to each incoming request.
- ✅ It appears in logs and traces across all services that handle the request.
- ✅ It helps you connect related log entries and traces, so you can follow one request from start to finish.

## Request Flow (End-to-End View)

**User Request** (e.g., `req-7f3a2e`) → **API** (Receives request, logs and forwards it) → **Backend** (Processes request, may call other services) → **Database** (Runs a query and returns data)

## Essential Commands

| Command | What It Does |
|---|---|
| `kubectl logs <pod>` | View logs from a Pod (e.g., kubectl logs api-7d9f8). |
| `kubectl describe pod <pod>` | Show detailed information about a Pod, including events, errors, and resource status. |

## Example: Correlating Logs with a Request ID

```
2026-11-01 10:24:11  INFO  [req-7f3a2e] Received request: GET /orders/123
2026-11-01 10:24:11  INFO  [req-7f3a2e] Calling database: SELECT * FROM orders WHERE id=123
2026-11-01 10:24:18  WARN  [req-7f3a2e] Database response took 7.2s
```

## Troubleshooting: Slow API Response

| Step | Action |
|---|---|
| 1 Confirm the issue | Check metrics or logs for higher response time (latency). |
| 2 Follow the request | Search logs/traces using the request ID (e.g., `req-7f3a2e`) across services. |
| 3 Identify the delay | See where the time is spent (e.g., database query took 7.2s). |
| 4 Fix the cause | Optimize the slow database query (e.g., add an index, review query plan, or increase resources). |

## Production Awareness

- Never log sensitive data such as passwords, tokens, API keys, or personal information (PII).
- Use log masking (e.g., ****) for any sensitive fields.
- Keep logs useful, structured, and with a request ID for easy investigation.

## Key Takeaway

- Use logs to see what happened and traces to see how a request moves across services.
- A request ID and good logging practices make production troubleshooting faster and safer.

# The Complete Toolchain: One Application, One Release

## What This Page Shows

- How the key DevOps tools work together to take one code change from development to production safely and reliably.
- A practical, end-to-end release flow for a small web application (e.g., a web API or website).

## Release Flow Summary

1. Developer commits a change and opens a pull request.
2. GitHub Actions runs tests and security checks.
3. Docker builds a versioned image.
4. Image is pushed to Amazon ECR.
5. Terraform manages the AWS infrastructure (e.g., EKS cluster, IAM).
6. Helm defines the application release (e.g., values, replicas, resources).
7. Argo CD syncs the approved change to deploy to Kubernetes.
8. Prometheus, Grafana, and logs verify the release in production.

## Tools and Their Roles

| Tool | Role in the Release |
|---|---|
| GitHub | Code hosting, pull requests, and code review. |
| GitHub Actions | Runs tests, security checks, and builds the app. |
| Docker | Builds a versioned container image (e.g., v1.2.3). |
| Amazon ECR | Stores the container image securely. |
| Terraform | Provisions and manages AWS infrastructure (e.g., VPC, EKS, IAM) as code. |
| Helm | Packages the Kubernetes release (app, config, values). |
| Argo CD | Pulls the approved config from Git and deploys to Kubernetes (GitOps). |
| Prometheus | Collects metrics from the running application. |
| Grafana | Visualizes metrics, builds dashboards, and shows health. |
| Logs | Application and system logs for troubleshooting (e.g., Loki, Elasticsearch). |

## End-to-End Release Flow: CI and CD Pipelines

### CI Pipeline (Build, Test, and Package)

Developer (Commit & Pull Request) → GitHub (Code & PR) → GitHub Actions (Run Tests & Security Checks) → Docker (Build Image (e.g., v1.2.3) → Amazon ECR (Store Versioned Image)

### CD Pipeline (Deploy and Run)

Terraform (Provision/Update AWS Infrastructure (e.g., EKS, VPC, IAM)) → Helm (Define Release: Chart + Values) → Argo CD (Sync & Deploy from Git to Kubernetes) → Kubernetes (Run Application in Pods)

### Verification (Observe)

Prometheus (Metrics) | Grafana (Dashboards) | Logs (Loki / ES) (Troubleshoot)

## Realistic Example: A Web App Release

```
# 1. Build & push image (CI)
docker build -t webapp:v1.2.3 .
docker tag webapp:v1.2.3
123456789012.dkr.ecr.us-east-1.amazonaws.com/webapp:v1.2.3
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/webapp:v1.2.3

# 2. Deploy with Helm (CD)
helm upgrade --install webapp ./chart \\
  --namespace prod -f values-prod.yaml
```

## Verification Checklist

- ✓ Deployment is synced in Argo CD.
- ✓ Pods are running and healthy (kubectl get pods).
- ✓ Application is responding (health check / endpoint).
- ✓ Metrics are normal (request rate, error rate, latency).
- ✓ Logs show no critical errors.

## Production Insight

- Every release needs verification (not just "deployed successfully").
- Have a practical recovery path (e.g., rollback to the previous version via Helm or Argo CD).
- Monitor key signals after release: errors, latency, and user impact.

## Key Takeaway

A reliable release connects code → automated build → container → infrastructure → deployment → monitoring, with verification and a clear recovery plan.

# DevOps Toolchain Incident: Find the Broken Stage

## The Incident Scenario

- A developer merges a change.
- CI passes and the container image is published to ECR.
- Argo CD shows the deployment has synchronized.
- However, users are receiving **HTTP 503** errors.

## Application Request Path (Production View)




## What We Need to Find Out

- [x] Where the request is failing (LB → Ingress → Service → Pods).
- [x] Whether the Pods are running and ready to receive traffic.
- [x] If the readiness probe is failing.
- [x] What the application logs show.
- [x] Why the pipeline succeeded but users still get HTTP 503.

## Investigation Steps and Commands

| Step | Command | Purpose |
|---|---|---|
| 1 Check Pods | `kubectl get pods` | Verify Pod status (Running, Ready, Restarts). |
| 2 Check Endpoints | `kubectl get endpointslices` | Confirm if the Service has healthy endpoints (Pod IPs). |
| 3 Inspect a Pod | `kubectl describe pod <pod>` | Check events, probe status, and failure messages. |
| 4 View Logs | `kubectl logs <pod>` | Read application logs for errors (e.g., startup or config). |
| 5 Check Probe | `kubectl describe pod <pod> \| grep -i readiness` | See the readiness probe configuration and failures. |

## Key Findings (Example from a Real Incident)

```
$ kubectl get pods
NAME                    READY   STATUS    RESTARTS
webapp-7d9f4-abcde      0/1     Running   3
$ kubectl get endpointslices
NAME                    ENDPOINTS
webapp-svc-xyz          <none>
$ kubectl describe pod webapp-7d9f4-abcde
Readiness probe failed: HTTP probe failed
with status code: 503
```

## Root Cause (Example)

- The application is running, but the readiness probe is failing.
- Because the Pod is not Ready, it is removed from the Service endpoints (Endpoints = <none>).
- The Load Balancer and Service have no healthy Pods to route traffic to, so users receive HTTP 503.

## Recovery Decision

- **Option A:** Repair the configuration (e.g., fix the readiness probe path or app startup issue) and redeploy.
- **Option B:** Roll back to a known-good release (e.g., previous image/Helm release) if a quick and safe recovery is needed.

## Verify the Recovery

- [x] Confirm Pods are Ready: `kubectl get pods`
- [x] Ensure Service has endpoints: `kubectl get endpointslices`
- [x] Check application health: `curl -I https://app.example.com`
- [x] Validate metrics and errors in Grafana (error rate down, request rate normal).

## Final Challenge

- **Identify the failing component:** _____
- **Evidence for your diagnosis:** _____
- **Command used to verify the fix:** _____

## Key Takeaway

- A successful pipeline and a synchronized deployment do not guarantee application availability.
- Always verify Pod readiness, Service endpoints, and application health after every release, and have a clear rollback plan.